

Nom : Allaoua

Prénom : Kenza

Groupe : A1

Compte Rendu du Mini-Projet :

1 □ _Objectif du projet

Le projet vise à créer un **analyseur lexical et syntaxique simplifié pour le langage C**, permettant :

- La **reconnaissance des tokens** (mots-clés, identifiants, nombres, opérateurs et symboles).
- La **vérification syntaxique** pour des constructions limitées du C, notamment les déclarations, affectations et boucles `do/while`.
- Une interface simple permettant à l'utilisateur de **tester différents programmes C**, soit des exemples prédéfinis, soit un programme saisi manuellement. Le projet est organisé en **4 classes principales** :

Architecture du projet

Le projet est organisé en **4 classes principales** :

2_1Token

- Contient deux champs : `type` (type du token) et `valeur` (contenu exact du token).
- Fournit une méthode `toString()` pour l'affichage clair des tokens.
- **Rôle** : représenter chaque élément lexical identifié dans le programme C.

2_2Analyseur Lexical

- **Responsabilité** : découper le programme C en tokens.
- **Fonctionnalités** :
 - Reconnaissance des **mots-clés, fonctions, identifiants et nombres**.
 - Reconnaissance des **symboles simples** (`+`, `-`, `*`, `/`, `;`, `{`, `}`...) et des **opérateurs doubles** (`==`, `!=`, `<=`, `>=`, `++`, `--`).
 - Suppression des commentaires `//` et `/* ... */`.
- **Méthode principale** : `analyserProgrammeTokens(String[] programme) →` retourne une `List<Token>`.

2_3 Analyse Syntaxique

- **Responsabilité :** vérifier que les tokens respectent la grammaire C simplifiée.

2_4La grammaire de la boucle dowhile en langage c :

Programme -> (int) (main) () () { Bloc }

Bloc -> { Instruction }

Instruction -> Declaration

| ReturnStatement

| DoWhile

| InstructionSimple

Declaration -> Type Identifiant [(= Expression)] (;)

Type -> (int) | (float) | (char)

InstructionSimple -> Identifiant (++) | Identifiant (--) | Identifiant (= Expression) (;)

| FonctionAppel (;)

| String (;)

FonctionAppel -> Fonction ({ ExpressionOuString })

ExpressionOuString -> Expression | String

DoWhile -> (do) BlocOuInstructionSimple (while) (Expression) (;)

BlocOuInstructionSimple -> { Bloc } | InstructionSimple

ReturnStatement -> (return) Expression (;)

Expression -> Operand { Operateur Operand }

Operand -> Identifiant | Nombre | String

Opérateur -> (+) | (-) | (*) | (/)

| (<) | (>)

| (=) | (==) | (!=) | (<=) | (>=)